ÉTUDE DE CAS 01 / APPLICATION WEB

KORTIZ

De l’idée d’une dépense partagée à un système collaboratif fiable

CARLOS DA COSTA

ÉTUDE DE CAS APPROFONDIE · ÉDITION REVUE ET AUGMENTÉE · OCTOBRE 2026

Architecture, modèles de données, calcul financier, collaboration, sécurité, tests et déploiement.

Note de l’auteur

J’ai choisi de présenter Kortiz comme une étude de cas complète : non seulement ce que l’utilisateur voit, mais les contraintes invisibles qui rendent une application collaborative cohérente. Ce document suit le raisonnement d’ingénierie de la définition du problème jusqu’aux vérifications et aux perspectives de maintenance.

Le périmètre décrit la version publique du code étudiée le 30 octobre 2026. Le dépôt contient un frontend majoritairement en HTML/CSS/JavaScript natif, un backend Firebase, des règles Firestore/Storage, des fonctions cloud et des tests. Je distingue explicitement ce dépôt de toute version éventuellement déployée sur Firebase Hosting.

> **Méthode de lecture**
>
> Je présente les raisonnements de conception et les arbitrages techniques à partir des fichiers et tests vérifiables du projet. Les sections distinguent ce qui est **implémenté dans le code**, ce qui a été **validé par des tests**, et ce qui reste **à confirmer sur les services réels**. La première personne décrit mon appropriation et mon analyse du travail ; elle ne prétend pas prouver que chaque ligne de code a été écrite sans assistance.

Sommaire détaillé

01 L’origine du projet et le besoin réel 3

02 Découper le système en responsabilités 4

03 Modéliser les données sans perdre leur sens 6

04 Faire des calculs monétaires corrects 7

05 Déclarer un remboursement sans simuler une banque 9

06 Faire vivre un groupe : invitations, échanges et votes 10

07 Planifier, récurrer et convertir 11

08 Concevoir une expérience accessible dans quatre langues 12

09 Sécurité : partir des menaces, pas des promesses 13

10 Problèmes rencontrés, réponses et limites 14

11 Tester : distinguer preuve et couverture 16

12 Livrer sans mettre les données en danger 18

13 Arbitrages, recul critique et suite de travail 19

14 Du besoin exprimé au cahier des charges vérifiable 20

15 Ledger comptable : préserver les centimes et expliquer le résultat 21

16 Transactions, concurrence et traitements réessayables 22

17 Cycle de vie des groupes, des membres et des créances 24

18 Internationalisation, notifications et états de l’interface 25

19 Architecture de test : exemples reproductibles et limites 26

20 Performance, volumes et coût d’exploitation 28

21 Journal de décisions : savoir défendre un compromis 29

22 Préparer une soutenance technique 30

23 Annexes : repères techniques et sources 31

01 L’origine du projet et le besoin réel

Kortiz est parti d’une question quotidienne : comment suivre les dépenses d’un groupe sans devoir reconstituer des conversations, des reçus et des calculs dispersés ?

Un problème de coordination plus que de paiement

Lorsque plusieurs personnes partagent des frais de logement, de voyage ou de sortie, la difficulté n’est pas uniquement de saisir un montant. Il faut savoir **qui a avancé l’argent, qui doit en supporter quelle part, quelle somme reste due et si un règlement a réellement été confirmé**. Les mêmes questions réapparaissent quand un membre rejoint ou quitte le groupe, quand une dépense est corrigée ou quand une opération récurrente se déclenche.

J’ai donc défini Kortiz comme un **registre collaboratif**, et non comme une banque ou un service de paiement. Cette frontière a structuré les choix suivants : les comptes sont liés à une identité, les dépenses et les soldes sont calculés, les remboursements sont des déclarations à valider, mais aucun euro n’est reçu ni transféré par l’application.

Les exigences que j’ai retenues

- Un utilisateur doit pouvoir créer un groupe, y inviter des personnes et comprendre ses droits.
- Une dépense doit identifier au moins son montant, ses payeurs, sa répartition et son contexte.
- Les comptes d’un groupe doivent demeurer cohérents malgré les arrondis et les modifications.
- La confirmation d’un remboursement doit être distincte d’une simple déclaration « j’ai payé ».
- Les prévisions doivent aider à anticiper, sans fabriquer de dette comptable.
- L’expérience doit fonctionner sur mobile et dans quatre langues : français, anglais, portugais et espagnol.
- La communication doit rester liée au groupe : discussions, messages privés, sondages et notifications configurables.

Transformer un usage en cas de test

J’ai privilégié les parcours représentatifs plutôt qu’une liste de boutons. Un scénario typique consiste à créer un groupe de trois personnes, enregistrer une dépense de 10,00 € payée par une seule personne, partager cette somme à parts égales, puis vérifier que la somme des parts vaut toujours 1 000 centimes et que les débiteurs peuvent identifier précisément leurs obligations.

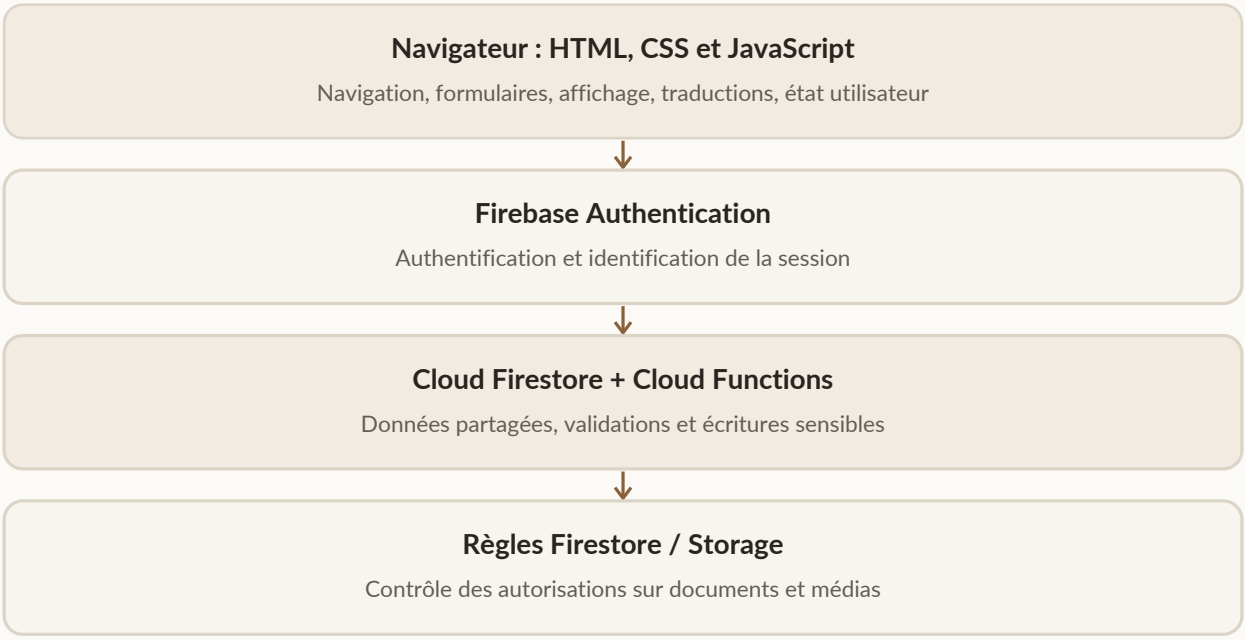
Cette démarche évite d’évaluer le produit uniquement sur son apparence : elle associe chaque fonctionnalité à un comportement attendu et à un invariant qui doit résister aux cas limites.

Ce que ce projet démontre

Un besoin utilisateur peut être transformé en modèles de données, règles métier, contrôles d'autorisation et tests reproductibles, sans confondre une interface convaincante avec une logique métier fiable.

02 Découper le système en responsabilités

Le premier arbitrage a été de séparer le code qui affiche les informations, celui qui calcule, celui qui autorise les opérations et celui qui persiste les données.



Pourquoi un frontend JavaScript natif ?

Le code public repose principalement sur **public/index.html, public/app.js et public/styles.css**. Les modules séparés pour les devises, le planning et les notifications fournissent des fonctions métier plus faciles à tester. J'assume ce choix comme un état technique réel : le frontend publié n'est pas une application React, même si une migration vers une architecture à composants pourrait devenir pertinente.

Ce découpage simplifie le chargement et évite une chaîne de compilation frontend complexe ; en contrepartie, un fichier central volumineux comme public/app.js demande une discipline particulière. Je peux expliquer les principaux sous-systèmes par leurs fonctions et leurs données plutôt que de présenter cette taille comme un idéal architectural.

Ce qui appartient au backend

Les fonctions définies dans **functions/index.js** prennent en charge des actions pour lesquelles une simple validation HTML serait insuffisante : adhésion au groupe, gestion de dépenses et de remboursements, opérations récurrentes, notifications ou suppression de compte. Le serveur reçoit un utilisateur authentifié, vérifie le contexte et ne doit autoriser que l'action permise.

Les règles **firestore.rules** et **storage.rules** définissent parallèlement les conditions de lecture et d'écriture. Cette architecture applique une idée fondamentale : masquer un bouton à un non-administrateur n'empêche pas un appel direct à la base ; l'autorisation doit être vérifiée au niveau du service.

Principes retenus dans le code

Responsabilité	Exemples vérifiables
Affichage	public/app.js, index.html, styles.css et pages publiques
Calcul réutilisable	public/currency-logic.js, planning-logic.js, notification-logic.js
Identité et données	Firebase Auth, SDK Firestore et collections de groupes
Traitement sensible	functions/index.js et utilitaires de calcul de dettes
Accès aux ressources	firestore.rules, storage.rules
Livraison	firebase.json, index Firestore, scripts et GitHub Actions

> **Compromis de maintenance**
>
> Une migration totale vers React/TypeScript ne serait pas seulement esthétique : elle devrait conserver les invariants financiers, les identifiants Firestore, les traductions, le formulaire de contact et les droits actuels. Je privilégierais donc une refonte par étapes vérifiées.

03 Modéliser les données sans perdre leur sens

Des entités distinctes

La logique distingue les **profils utilisateurs**, les **groupes**, les **membres**, les **dépenses**, les **règlements déclarés**, les **prévisions**, les **invitations** et les objets de **messagerie/sondage**. Cette séparation me permet de répondre à des questions métier différentes sans faire d'un message de chat une écriture comptable.

Une dépense porte par exemple des montants en centimes, une table des payeurs et une répartition par participant ; le registre des règlements contient, lui, la direction et la valeur d'une compensation. Une invitation et un rôle administratif relèvent des autorisations, pas des soldes.

Pourquoi ne pas enregistrer un solde définitif ?

Le code de **computeBalances** recalcule les soldes à partir des dépenses et des règlements. Ce choix privilégie une source de vérité reconstituable : si une dépense autorisée est modifiée, le résultat doit pouvoir être recalculé. Le solde net d'un membre suit la relation **net = payé − dû + envoyé − reçu**.

L'autre option aurait été de maintenir un champ solde modifié à chaque événement. Elle aurait demandé davantage de garanties transactionnelles et créé un risque de dérive lorsqu'un document est corrigé, refusé ou rejoué. Recalculer ne supprime pas tous les problèmes, mais rend le raisonnement plus traçable.

```
pour chaque membre : net = 0
pour chaque dépense non rejetée :
net[payeur] += montant payé
net[participant] -= part due
pour chaque règlement confirmé :
net[émetteur] += montant
net[destinataire] -= montant
vérifier : somme(net) = 0
```

Une frontière essentielle : prévoir n'est pas dépenser

Le module **planning-logic.js** calcule séparément les dépenses réalisées, les dépenses prévues et leur projection budgétaire. Le résultat comprend le budget éventuel, le reste prévisionnel, le ratio utilisé et un indicateur de dépassement. Les prévisions ne doivent jamais augmenter automatiquement une dette réelle.

Cette séparation paraît simple, mais elle évite une erreur de produit fréquente : un montant saisi pour anticiper une dépense serait interprété comme une obligation entre membres avant même qu'un paiement n'existe.

Compatibilité avec des versions précédentes

Les règles métier documentent un cas de migration : d'anciennes dépenses au statut **pending** doivent être prises en compte sans réécrire tous les documents historiques, tandis que les dépenses **rejected** restent exclues. Je retiens ici le principe de compatibilité descendante : faire évoluer l'interprétation d'un document peut être moins risqué que modifier immédiatement les anciennes données.

Il faut toutefois vérifier que le backend, le frontend et les règles publiés sont alignés. Un changement de logique de comptabilisation publié d'un seul côté peut produire des soldes divergents.

04 Faire des calculs monétaires corrects

Pour un outil de dépenses partagées, la première exigence algorithmique est de ne pas perdre un centime à cause des arrondis.

Travailler en entiers plutôt qu'en nombres flottants

Le code convertit les montants affichés en **centimes entiers**. La fonction de saisie accepte différents séparateurs décimaux et rejette les valeurs invalides. L'arithmétique des répartitions se fait sur des entiers, ce qui évite d'accumuler des écarts comme 0,1 + 0,2 en virgule flottante.

Les formats d'entrée sont adaptés aux habitudes locales : la virgule et le point peuvent représenter des décimales ou des séparateurs de milliers. Cette souplesse impose de valider rigoureusement les chaînes ambiguës avant de les convertir.

Algorithme du plus fort reste

Pour partager 10,00 € entre trois participants, diviser 1 000 par 3 conduit à 333,333… centimes. Le système calcule d'abord la partie entière de chaque part, puis répartit les centimes restants aux participants ayant les plus grands restes. À égalité, l'ordre des participants départage les résultats pour garantir un calcul déterministe.

Dans le code, l'algorithme **allocate(total, weights)** généralise cette approche à des poids. Il est utilisé pour les répartitions égales, proportionnelles aux parts et proportionnelles aux pourcentages. Il garantit que la somme des sorties est exactement le total, sous réserve d'entrées valides.

```
Montant : 1 000 centimes, poids = [1, 1, 1]
Parties entières : [333, 333, 333]
Reste à attribuer : 1 centime
Résultat déterministe : [334, 333, 333]
Invariant : 334 + 333 + 333 = 1 000
```

Quatre modes de répartition

Mode	Vérification essentielle
Égal	Tous les participants ont le même poids.
Parts	Les poids doivent être strictement positifs.
Pourcentages	Le total doit atteindre exactement 100 %, représenté en centièmes de pour cent.
Montants exacts	La somme des montants saisis doit correspondre à la dépense.

Simplifier les remboursements proposés

Le calcul **minimalTransfers** sépare créanciers et débiteurs, les trie par montant décroissant puis associe successivement les valeurs les plus importantes. Ce mécanisme produit une proposition simple de transferts à effectuer **hors application** ; il ne lance aucune transaction.

Je le présenterais comme une **heuristique gloutonne** de règlement : elle est lisible et pratique, mais l'objectif mathématique de minimiser globalement le nombre de transactions dans tous les cas demande une analyse spécifique. Je n'affirme pas que cet algorithme résout toutes les variantes d'optimisation.

> **Test d'invariant majeur**
>
> Pour tout groupe cohérent, la somme algébrique des soldes nets doit valoir zéro. Chaque dépense ajoute autant de valeur dans « payé » que dans « dû » ; chaque règlement transfère une valeur entre deux personnes sans créer ni détruire un montant.

05 Déclarer un remboursement sans simuler une banque

L’état « payé » ne suffit pas

Une difficulté métier importante consiste à traiter un utilisateur qui déclare avoir payé sans que l’application ait la preuve bancaire du transfert. La solution documentée sépare la **demande ou déclaration du débiteur** de la **confirmation par un administrateur**. La première n’altère pas les soldes ; la confirmation fait entrer un règlement dans le calcul.

J’ai retenu une règle de prudence : le logiciel n’affiche pas une déclaration comme un transfert bancaire vérifié. C’est également pour cette raison que les moyens de remboursement éventuels (coordonnées SEPA ou Wero) sont présentés comme des indications pour une action effectuée ailleurs.

IBAN, QR SEPA et contact Wero

Le code contient une validation d’IBAN fondée sur la structure, certaines longueurs nationales et la **clé de contrôle modulo 97**. Il génère également une charge utile de QR SEPA au format EPC et vérifie la longueur produite. Ces traitements améliorent la qualité d’une information de destinataire, mais n’authentifient pas la propriété du compte bancaire.

La normalisation d’un contact Wero différencie e-mail et téléphone. Là encore, il s’agit d’un champ d’information et non d’une intégration de paiement. Cette nuance réduit le risque de produire une promesse fonctionnelle ou réglementaire inexacte.

Le problème des dettes déjà déclarées

Dans les fonctions de sortie de groupe, **computeAvailableTransfers** prend les dépenses, règlements et créances de sortie encore ouvertes. Une créance ouverte réserve une partie des soldes correspondants afin de limiter les déclarations multiples sur la même dette.

Cette conception montre qu’un état métier a des conséquences : une demande « en attente » n’est pas une dépense et ne peut pas être traitée comme un règlement confirmé, mais elle peut déjà réserver la dette pour empêcher des doublons.

> **Limite structurante**
>
> Si une dépense ancienne est modifiée après un remboursement confirmé, une réconciliation peut être nécessaire. Le code et la documentation signalent ce risque ; un historique d’audit aide à le détecter, mais un verrouillage ou un mécanisme de correction comptable serait une évolution importante.

06 Faire vivre un groupe : invitations, échanges et votes

Inviter et vérifier les droits

Le groupe doit rester un espace fermé à ses membres, même lorsque son code d'invitation circule. Le code génère des identifiants avec **crypto.getRandomValues**, les normalise et utilise des opérations serveur pour rejoindre un groupe. Cela évite de faire de la possession d'un lien une permission illimitée.

Les opérations côté serveur emploient des fonctions de type **getGroupFor**, **isAdmin** et des validations d'identifiants. Je considère la vérification de l'appartenance comme une condition répétée à chaque action sensible, et non comme une étape unique lors de l'ouverture du groupe.

Messages privés, discussions et blocages

Les messages privés sont représentés par des conversations entre membres. Le backend construit un identifiant déterministe à partir des deux identités et vérifie qu'une conversation contient bien les participants attendus. Les règles de blocage sont vérifiées pour les deux directions.

Cela répond à un problème d'abus et de confidentialité : un utilisateur doit pouvoir empêcher une interaction non souhaitée ; le frontend n'est pas la source de vérité sur l'existence et l'autorisation d'une discussion.

Sondages intégrés aux conversations

La version documentée place les sondages directement dans le fil de discussion, aussi bien pour les groupes que pour les conversations privées. Le modèle doit distinguer la question, les options, les votes et l'état de clôture. Il faut empêcher qu'un membre extérieur puisse lire ou modifier des votes.

Le choix de conserver les sondages dans le flux plutôt que dans un onglet isolé répond à un problème d'usage : une décision collective fait partie de la discussion. Il introduit néanmoins des questions de pagination et d'écoute temps réel ; la documentation signale une limite de requête sur les 80 messages récents par conversation pour certains flux de sondages.

Journaliser sans surcollecter

Les activités importantes, comme la création d'une dépense récurrente, peuvent produire des événements de journal. Ces traces doivent rester proportionnées : utiles pour comprendre les opérations, mais jamais un prétexte pour publier des conversations ou des données privées dans des logs de développement.

07 Planifier, récurrer et convertir

L'application ne se limite pas à l'instant où une dépense est enregistrée : elle aide aussi à anticiper et à présenter des soldes dans une devise d'affichage.

Dépenses récurrentes et idempotence

Les récurrences hebdomadaires, mensuelles ou annuelles posent un problème de dates. Une échéance fixée au 31 ne peut pas exister dans tous les mois. La fonction **addPeriod** calcule la prochaine date en tenant compte du nombre de jours du mois et de la date d'origine, plutôt qu'en ajoutant naïvement 30 jours.

Le backend **recurringExpenses** utilise une tâche planifiée et, pour les écritures, des transactions Firestore. Les identifiants déterministes construits avec la date de l'occurrence permettent de reconnaître une opération déjà créée. L'objectif est de rendre les reprises et les déclenchements multiples **idempotents** : une réexécution ne doit pas dupliquer une dépense.

Traiter les erreurs sans bloquer toutes les récurrences

Le traitement parcourt les échéances et isole les erreurs de chaque document. Une récurrence endommagée ne doit pas empêcher la mise à jour des autres groupes. C'est une distinction essentielle entre robustesse et silence : continuer le traitement est utile, mais chaque échec doit aussi être signalé par une trace adaptée.

Devises : séparer comptabilité et présentation

Le module **currency-logic.js** convertit les soldes pour leur affichage, en passant par des taux de change validés. Il rejette un taux négatif, infini ou absent, et renvoie une absence de résultat si la conversion n'est pas possible.

Je ne transforme pas la devise de stockage d'une dépense en fonction du tableau de bord consulté. La conversion relève de la **visualisation** ; il faut conserver la trace de la devise du groupe et éviter de présenter un taux périmé comme un taux certain.

Notifications maîtrisées

Les préférences de notifications sont organisées par catégorie : invitations, discussions, nouvelles dépenses, demandes et confirmations. L'interface peut aussi mettre un groupe en sourdine. Le Service Worker Firebase sert de point d'intégration aux notifications lorsque l'environnement et le navigateur l'autorisent.

Une notification ne doit jamais devenir une condition de validité d'une opération métier. Un compte doit pouvoir consulter l'état réel d'un remboursement même si sa notification push a été refusée, perdue ou désactivée.

08 Concevoir une expérience accessible dans quatre langues

Internationalisation

Le fichier **public/i18n.js** porte les textes et la sélection de langue pour les interfaces FR/EN/PT/ES. La langue peut suivre la préférence de l'utilisateur ou celle du navigateur. Les messages dynamiques et les textes de formulaires doivent être traduits, pas seulement les rubriques visibles au premier chargement.

La localisation touche aussi la saisie monétaire, le format de devise et la lecture de dates. J'ai donc traité les traductions comme une contrainte fonctionnelle transversale : un avertissement de sécurité absent en portugais pourrait provoquer une incompréhension aussi concrète qu'un bouton mal placé.

Responsive et navigation

Les fichiers CSS et les tests sur les éléments de navigation montrent un travail de stabilisation mobile. Une interface de groupe chargée peut rapidement devenir illisible sur écran étroit. Le choix documenté de cinq entrées principales — dépenses, prévisions, discussion, soldes et plus — rend les fonctions centrales accessibles sans exposer toutes les options simultanément.

Dans le parcours public, les pages de présentation, l'écran d'accueil et les paramètres doivent rester lisibles sur desktop, tablette et mobile. Les tests statiques ne garantissent pas à eux seuls l'absence de défauts sur un appareil réel ; une revue visuelle au clavier et au tactile demeure nécessaire.

Formulaire de contact et cadre juridique

Les pages publiques comprennent le contact et les informations légales. Le dépôt cherche à préserver le formulaire opérationnel lors des évolutions. Les messages de contact ne doivent pas être confondus avec les conversations d'utilisateurs ni avec une notification push.

J'en retiens qu'une refonte visuelle peut casser un comportement métier simple mais important. Lorsqu'un flux fonctionne, je le protège par des vérifications ciblées au lieu de le réécrire sans raison. Les mentions RGPD et l'information de l'utilisateur nécessitent en outre une validation juridique distincte.

09 Sécurité : partir des menaces, pas des promesses

Définir le modèle de menace

Les risques prioritaires sont l'accès d'un membre à un autre groupe, l'usurpation d'un rôle administrateur, la modification frauduleuse d'un montant, la divulgation de messages privés, l'exploitation abusive des fonctions et les écritures concurrentes. Une application qui traite des dettes déclaratives doit être particulièrement prudente sur la notion d'autorité.

Je distingue les validations d'interface, les règles d'accès Firestore et les contrôles backend. Elles se renforcent mutuellement, mais ne sont pas interchangeables.

Contrôles vérifiables

- La règle est de vérifier l'identité authentifiée avant les mutations sensibles et l'appartenance au groupe concerné.
- Les règles Firestore portent des fonctions de validation des champs, des montants et des permissions.
- Les montants sont bornés et les identifiants sont validés côté service.
- Les opérations sensibles de la messagerie et des dépenses passent par des chemins contrôlés.
- Le dépôt comprend un scanner de secrets et un fichier SECURITY.md destiné aux signalements privés.
- Les en-têtes de sécurité HTTP et la CSP sont configurés dans firebase.json et doivent être validés après publication.

Ce qu'il ne faut pas prétendre

Une clé Firebase publique intégrée au client n'est pas une clé de compte de service. Sa présence n'est pas une fuite en soi ; la sécurité repose sur les règles et les contrôles effectifs. À l'inverse, un jeton privé, une clé de service ou une variable d'environnement backend sensible ne doit jamais être publié.

La présence de règles dans le dépôt ne démontre pas qu'elles sont déployées à l'identique. Je dois vérifier la version réellement active dans la console, puis tester avec au moins deux comptes et des rôles distincts. Un scanner qui ne trouve rien n'est pas une garantie absolue.

> **Principe de moindre privilège**
>
> Une fonctionnalité disponible visuellement n'accorde aucun droit. Le service vérifie qui appelle, sur quelle ressource et dans quel état de groupe. En cas de doute, l'action est refusée plutôt que déduite du contexte affiché.

10 Problèmes rencontrés, réponses et limites

Un projet progresse lorsque les cas difficiles deviennent des règles métier explicites, des vérifications ou des décisions d'architecture. Les cas suivants sont attestés par la logique ou la documentation du dépôt.

Les centimes manquants dans un partage

Contexte et problème. Un partage décimal ordinaire peut perdre un centime lorsqu'on arrondit chaque personne indépendamment.

Analyse. Une simple division suivie d'un arrondi local ne garantit pas la conservation du total.

Réponse technique. J'utilise une répartition en centimes par méthode du plus fort reste et départage déterministe.

Vérification. Les fonctions de partage et les tests de logique peuvent vérifier la conservation de la somme.

Ce que je retiens. Le résultat exact peut différer d'un centime entre participants, mais l'écart est explicable et stable.

Une déclaration de paiement prise pour une confirmation

Contexte et problème. Un utilisateur peut cliquer « j'ai payé » alors que l'application n'a aucune visibilité sur le transfert réel.

Analyse. Une déclaration ne représente pas une preuve de paiement.

Réponse technique. La logique distingue demande, état déclaré et confirmation par un rôle autorisé.

Vérification. Les scénarios décrivent un débiteur déclarant un paiement, un membre non-admin refusé et un admin confirmant.

Ce que je retiens. Même confirmée dans l'application, l'écriture reste une confirmation humaine hors paiement intégré.

Une ancienne dépense pending oubliée

Contexte et problème. Une évolution de la validation a fait apparaître des documents au statut historique pending.

Analyse. Une lecture ne considérant que approved peut écarter à tort ces dépenses.

Réponse technique. Le comportement documenté compte les dépenses non rejetées et conserve les documents historiques.

Vérification. Les tests dédiés de politique de dépenses et de régression financière ciblent les statuts et les soldes.

Ce que je retiens. Vérifier l'alignement de toutes les fonctions et des règles réellement publiées.

Des récurrences dupliquées après reprise

Contexte et problème. Une tâche planifiée peut être relancée ou interrompue entre deux écritures.

Analyse. Sans clé stable ni transaction, une occurrence peut être créée plusieurs fois.

Réponse technique. Des identifiants déterministes par date et des transactions Firestore rendent le traitement réessayable.

Vérification. Vérification des dates attendues, des IDs et de l'unicité dans le code/test concernés.

Ce que je retiens. Le comportement réel dépend aussi du scheduler, des quotas et des conditions de production.

L'utilisateur quitte le groupe avec une dette

Contexte et problème. Une sortie ou suppression de compte ne doit ni effacer la dette des autres ni permettre de réserver deux fois le même montant.

Analyse. Les écritures de départ et les créances ouvertes nécessitent une représentation distincte des paiements effectifs.

Réponse technique. Le calcul de transfert disponible tient compte des créances de sortie encore ouvertes et des soldes résiduels.

Vérification. Tests de dettes de sortie et de garde-fous de suppression de compte.

Ce que je retiens. Les opérations irréversibles restent à tester avec des comptes réels et des scénarios de concurrence.

Les messages privés exposés à un tiers

Contexte et problème. Un identifiant de conversation devinable ne doit pas suffire à la lire.

Analyse. Les contrôles frontend sont contournables.

Réponse technique. Le backend vérifie les deux participants et les blocages, avec des règles Firestore associées.

Vérification. Tests de références de chat, de contrôles d'accès et de messages privés.

Ce que je retiens. L'audit de la configuration déployée reste indispensable.

Conversion monétaire incomplète

Contexte et problème. Tous les taux de change ne sont pas toujours disponibles ou à jour.

Analyse. Afficher un résultat par défaut arbitraire serait trompeur.

Réponse technique. La conversion retourne null en cas de données insuffisantes et signale les devises indisponibles.

Vérification. Tests de conversion et de résumé multi-groupes.

Ce que je retiens. Une conversion d'affichage n'est pas une cotation bancaire exécutable.

Déploiement partiel frontend / backend

Contexte et problème. Une nouvelle interface peut parler à une ancienne fonction avec un contrat différent.

Analyse. Le risque est une incohérence de traitement des dépenses ou des permissions.

Réponse technique. La documentation impose sauvegarde, comparaison des règles, tests multi-comptes et ordre de déploiement contrôlé.

Vérification. Vérification des scripts, des fonctions et des scénarios avant publication.

Ce que je retiens. La suite locale ne vaut pas un test de production.

11 Tester : distinguer preuve et couverture

Stratégie de non-régression

La copie de code public a donné un résultat documenté de **257 tests réussis sur 257** lors de sa préparation. Les GitHub Actions récents de la branche principale ont eux aussi conclu à un succès pour le workflow de contrôles. J'utilise ces résultats comme indication de reproductibilité, sans les transformer en promesse d'absence de bugs.

La suite s'appuie principalement sur le test runner Node et des vérifications métier, statiques ou par simulation. Elle couvre des invariants financiers, les accès, les modules de planification, le responsive, l'internationalisation et plusieurs parcours sociaux. Certains tests inspectent aussi le code source ou des fragments ; il faut savoir dire exactement ce qu'ils prouvent.

Famille de tests	Ce que je cherche à démontrer
Calculs	Sommes de parts, soldes nets, demandes déclaratives, récurrences et sorties.
Accès	Refus hors groupe, permissions admin, protections de messagerie et sondages.

Famille de tests	Ce que je cherche à démontrer
Interface	Présence des éléments attendus et cohérence de certaines navigations mobiles.
Internationalisation	Présence des traductions et gestion de chaînes dynamiques.
DevOps	Exécution Node et recherche de secrets dans GitHub Actions.

Mon protocole manuel idéal avec deux comptes

- Créer deux comptes A et B et un groupe privé auquel A invite B.
- Faire accéder un compte C non membre aux identifiants de groupe et de message ; vérifier le refus côté service.
- Saisir une dépense de 10,00 € partagée, comparer les résultats entre A et B et contrôler les invariants en centimes.
- Déclarer un remboursement sans validation, vérifier que le solde n'a pas bougé, puis confirmer avec l'administrateur.
- Créer et clôturer un sondage, puis tester les limites de vote depuis chaque rôle.
- Rejouer une récurrence et vérifier l'absence de doublon ; tester les dates de fin de mois.
- Déconnecter le réseau, recharger les données, changer la langue et tester l'affichage sur mobile.
- Documenter toute anomalie observée et reproduire la correction avec un test adapté.

Ce que la CI n'exécute pas

Le workflow du dépôt lance la suite Node et un scanner de secrets. Il ne provisionne pas automatiquement un vrai Firestore avec plusieurs comptes, ne teste pas l'ensemble des interfaces sur de vrais mobiles et n'atteste pas de la conformité juridique. Je ne présenterais donc pas 257 tests comme une couverture de 100 % du produit.

> **Formulation rigoureuse pour l'oral**
>
> « Les tests automatisés constituent un filet de sécurité. J'explique lesquels valident la logique pure, lesquels inspectent la structure du code, et lesquels exigent encore un navigateur et un environnement Firebase réels. »

12 Livrer sans mettre les données en danger

Versionner uniquement ce qui doit l'être

Le dépôt conserve le code client, les fonctions, les règles, les index et les scripts de vérification. Il doit exclure les fichiers de secrets, exports Firestore, jetons et caches locaux. La publication GitHub de la source et le déploiement Firebase sont deux actes différents : l'un expose du code, l'autre modifie un service utilisé.

J'ai retenu des garde-fous contre les déploiements involontaires et une procédure de revue de configuration. Les scripts historiques sont utiles comme éléments de traçabilité, mais ils peuvent contenir des références à d'anciennes versions ; ils ne doivent pas remplacer la vérification manuelle du contexte.

Ordre de publication sensible

Pour l'évolution des permissions et de la suppression de compte, la documentation propose une séquence conditionnée à la compatibilité : sauvegarder et comparer les règles actuelles, tester, puis déployer les règles, la fonction deleteAccount et enfin Hosting. Pour une autre modification de politique financière, les dépendances entre fonctions et frontend peuvent imposer une séquence différente.

La règle générale n'est donc pas « toujours déployer les mêmes éléments dans le même ordre », mais **déterminer les dépendances de contrat et préparer un retour arrière**. Une version partiellement publiée peut être plus dangereuse qu'une version ancienne mais cohérente.

Intégration continue et évolutions

Le dépôt contient un workflow GitHub Actions de tests et un fichier de configuration Dependabot pour signaler les mises à jour de dépendances. Les Pull Requests de changement majeur doivent être étudiées à partir des contrats Firebase utilisés et des résultats CI, sans fusion automatique.

L'absence de licence open source explicite signifie également que la publication du code ne constitue pas une autorisation générale de réutilisation. Cette décision doit être documentée au même titre que les choix d'hébergement.

13 Arbitrages, recul critique et suite de travail

Ce que je considère comme solide

- Une frontière claire entre dépenses, remboursements déclaratifs et transferts bancaires réels.
- Des montants stockés en centimes et des répartitions contrôlables par invariants.
- Une séparation des rôles entre l'interface, les fonctions sensibles et les règles de données.
- Des modules de calcul isolés et une suite de non-régression automatisée.
- Un travail explicite sur le multilingue, les appareils mobiles et la confidentialité.

Ce qui me paraît encore fragile

Le frontend centralisé dans app.js gagnerait à être découpé davantage, avec des contrats typés entre les modules et les fonctions. L'observabilité de production pourrait être renforcée, tout en gardant les journaux expurgés. Les tests d'intégration multi-comptes restent une priorité : une règle qui paraît correcte dans un fichier peut se comporter différemment lorsqu'elle interagit avec des requêtes réelles.

La correction de dépenses anciennes après confirmation de règlements exige un modèle d'audit et de réconciliation particulièrement prudent. Les quotas Firebase et les migrations de règles ne sont pas seulement des problèmes de configuration : ils influencent les possibilités de livraison.

Feuille de route réaliste

Priorité	Étape mesurable
1	Mettre en place un projet Firebase de test et exécuter les scénarios multi-comptes.
2	Ajouter des tests d'intégration des règles et des écritures backend.
3	Distinguer clairement les écritures de correction et les règlements déjà confirmés.
4	Isoler progressivement les responsabilités de public/app.js et documenter leurs contrats.
5	Faire une revue d'accessibilité et de performance mobile ; conserver des résultats datés.

14 Du besoin exprimé au cahier des charges vérifiable

La qualité d'une application ne dépend pas du nombre de fonctionnalités affichées dans son menu, mais de la précision avec laquelle elle définit les opérations qu'un utilisateur peut effectuer et les situations qu'elle refuse.

Raisonner en parcours plutôt qu'en écrans

Je peux décrire le parcours minimal par une suite de responsabilités : créer un compte, définir son identité, ouvrir un groupe, en partager l'accès, saisir une dépense, laisser le système calculer les parts et obtenir une confirmation de règlement. Chaque action modifie un type de donnée différent. Cette description révèle tout de suite les cas qui dépassent l'interface : un compte supprimé, un membre qui quitte le groupe, un document déjà approuvé ou une tentative de modification par une personne non autorisée.

Le travail de conception consiste à formuler des critères d'acceptation indépendants du rendu graphique. Une dépense rejetée ne doit pas entrer dans les soldes. Une prévision ne doit pas devenir une dette. Un bouton de remboursement ne doit pas être assimilé à un virement. Un invité ne doit pas pouvoir accéder à la conversation avant que ses droits aient été établis. Ces règles donnent un sens aux tests et évitent de corriger des symptômes sans comprendre la cause.

Besoin utilisateur	Invariant / critère d'acceptation	Source observable
Partager une dépense	La somme des parts est égale au total de la dépense.	tests/logic.test.js
Consulter un budget	Les prévisions ne modifient pas les créances.	public/planning-logic.js
Changer la devise	La conversion reste une valeur d'affichage.	public/currency-logic.js
Quitter un groupe	Une créance ouverte n'est ni perdue ni comptée deux fois.	functions/exit-debt-logic.js
Recevoir des notifications	Les événements personnels et anciens sont filtrés.	public/notification-logic.js

Pourquoi formaliser avant de multiplier les écrans

Une interface peut paraître complète et présenter pourtant deux représentations contradictoires de la même dette. Formaliser les invariants m'oblige à décider qui fait autorité et à reconnaître que certaines informations n'ont pas le même statut : une estimation, une déclaration de paiement et une confirmation sont trois choses différentes. Cette clarification rend également le dialogue avec des utilisateurs plus simple : je peux expliquer ce que Kortiz garantit, mais aussi ce qu'il ne peut pas savoir.

> **Position défendable à l'oral**
>
> Je présente le produit comme un problème de cohérence de données et de coordination. La maquette est importante ; elle ne peut pas remplacer les règles métier explicites et reproductibles.

15 Ledger comptable : préserver les centimes et expliquer le résultat

Une égalité de conservation

Le principe le plus important de la répartition est une conservation : pour chaque dépense, la somme de ce que les membres doivent assumer doit être strictement égale au montant financé. Je travaille donc avec des entiers représentant des centimes, car les décimaux binaires introduisent des erreurs difficiles à justifier. Le même invariant s'applique au groupe : la somme de tous les soldes nets doit être nulle lorsque toutes les lignes sont prises en compte.

Un partage de 10,00 euros entre trois personnes illustre le problème. Dix euros représentent 1 000 centimes. Une division uniforme produit 333,33 centimes environ, une valeur impossible à enregistrer comme somme entière. Je peux attribuer 334, 333 et 333 centimes ; le total reste de 1 000. Le choix du bénéficiaire du centime supplémentaire doit être déterministe afin que le résultat ne change pas à chaque calcul.

Membre	Montant payé	Part due	Solde avant remboursement
A	1 000 c	334 c	+666 c
B	0 c	333 c	-333 c
C	0 c	333 c	-333 c
Total	1 000 c	1 000 c	0 c

Les signes comptables, sans ambiguïté

Un solde positif signifie qu'un membre a avancé davantage que sa part et doit recevoir de l'argent ; un solde négatif signifie qu'il a consommé davantage qu'il n'a avancé. Le calcul part des paiements et des parts, puis intègre les remboursements constatés. Dans la logique réutilisable, un règlement diminue la dette de l'émetteur et la créance du destinataire en conservant la somme nulle.

J'accorde une importance particulière aux conventions de signe et aux transitions d'état. Une fonction mathématiquement correcte peut produire un comportement métier erroné si elle inclut une dépense rejetée ou une promesse de paiement comme si elle était exécutée. Les tests de régression doivent associer chaque calcul à son statut de validité.

```
Propriété à tester pour chaque groupe :
somme(payeurs) = somme(parts)
somme(solde_net_des_membres) = 0
solde(dette reglee) = solde(avant) + reglement
solde(creance reglee) = solde(avant) - reglement
```

Pourquoi un algorithme de transferts est une proposition, pas un paiement

Lorsque plusieurs créanciers et débiteurs coexistent, les transferts proposés résultent du rapprochement des soldes. Dans exit-debt-logic.js, le programme choisit les montants positifs et négatifs puis associe débiteurs et créanciers en donnant priorité aux plus grands montants. Ce calcul optimise la lisibilité des régularisations possibles, mais il ne réalise aucune transaction bancaire. Le fonctionnement final dépend de confirmations humaines externes.

16 Transactions, concurrence et traitements réessayables

La différence entre une fonction qui marche une fois et une fonction robuste

Une interface mono-utilisateur masque des difficultés classiques : deux membres peuvent cliquer simultanément, une requête peut être envoyée deux fois après une perte réseau ou une tâche planifiée peut redémarrer. Si le code se contente de lire un document puis d'écrire sans condition, les opérations peuvent se chevaucher. La bonne question n'est pas « est-ce que le scénario heureux passe ? », mais « que se passe-t-il lorsqu'un état change entre la lecture et l'écriture ? ».

La gestion des récurrences fournit un cas concret. Le test security_recurring_atomic.test.js simule des transactions sur une petite base en mémoire, avec des révisions de documents et la détection de conflits. Il vérifie la création d'une seule occurrence, le rattrapage limité, le comportement des groupes archivés et l'arrêt lorsque le participant a quitté le groupe. Il ne s'agit pas d'une preuve d'intégration à Firestore réel, mais d'un test plus utile qu'une simple recherche de texte dans les fichiers.

Stratégie : identifiants stables et transactions

L'identifiant d'une occurrence périodique peut dépendre du modèle et de la date. Un même calcul reproduit alors la même clé lors d'une nouvelle tentative : on réduit le risque de créer deux documents distincts pour la même échéance. La transaction permet de vérifier que la configuration du modèle n'a pas changé pendant l'opération. Ces deux mécanismes répondent à des risques complémentaires et doivent être testés ensemble.

Mode de panne	Conséquence sans garde-fou	Réponse recherchée
Deux exécutions du scheduler	Deux dépenses pour une échéance	ID déterministe et transaction
Changement du modèle pendant un calcul	Création avec des participants périmés	Relecture transactionnelle
Requête répétée par le navigateur	Écriture ou demande dupliquée	Vérification d'état et idempotence
Échec au milieu de la suppression	Données incohérentes ou créances orphelines	Préconditions et ordre d'opérations
Quota réseau dépassé	Vue vide ou action non confirmée	Erreur explicite et reprise prudente

Différence essentielle

L'idempotence signifie que répéter une même opération ne doit pas multiplier ses effets. Elle est différente de la simple gestion d'erreur : il faut pouvoir expliquer pourquoi la deuxième tentative ne crée pas une nouvelle dette.

17 Cycle de vie des groupes, des membres et des créances

L’appartenance comme propriété métier

Les groupes impliquent plus qu’une liste de noms. Une invitation peut être proposée, acceptée, refusée ou devenir invalide ; un membre peut perdre l’accès ; un administrateur peut être tenu de confirmer une opération. Je dois donc raisonner à partir des identifiants de compte et des documents réellement autorisés, pas seulement des éléments visibles dans la barre latérale. Les fonctions backend et les règles de sécurité sont conçues pour vérifier les conditions d’appartenance avant les accès sensibles.

Le départ d’un membre est l’une des difficultés les plus significatives. Effacer immédiatement toute information associée ferait perdre la traçabilité des dettes historiques. A contrario, laisser une créance de sortie ouverte tout en proposant la même somme comme remboursement ordinaire créerait un double comptage. Le module exit-debt-logic.js réserve les montants des créances de sortie ouvertes dans le calcul des transferts disponibles.

Suppression de compte et distinction des responsabilités

account-deletion-guard.js expose des fonctions de détection de dettes au niveau groupe, en messages privés et lors de sorties. Cette logique pure peut être appelée et testée indépendamment de la fonction Firebase qui exécute effectivement la suppression. C’est un bon exemple de séparation entre une règle métier — « cette suppression est-elle compatible avec les engagements courants ? » — et l’acte technique irréversible.

Je peux défendre une décision prudente : la vie privée impose de prévoir la suppression des données personnelles, mais la suppression ne doit pas être présentée comme un effacement magique des obligations financières reconnues par d’autres membres. Les deux objectifs sont à concilier dans des parcours explicites, avec anonymisation adaptée lorsque le modèle le permet.

Matrice de scénarios

Scénario	Règle attendue	Validation indicative
Invitation non acceptée	Pas de lecture du contenu privé du groupe	Test multi-comptes Firebase à prévoir
Membre sortant débiteur	Créance suivie sans double imputation	exit-debt-logic.js
Compte avec demande privée reconnue	Blocage ou traitement explicite	account-deletion-guard.js
Demande non acceptée	Ne pas transformer une requête en dette reconnue	directDebtsFor
Ex-membre tentant une écriture	Rejet côté règles / backend	Tests réels de permissions à compléter

18 Internationalisation, notifications et états de l’interface

Traduire des actions, pas seulement des étiquettes

Quatre langues créent une difficulté de maintenance : chaque message de confirmation, d’erreur, de chargement, de refus ou de réussite doit être cohérent avec le parcours. La base de traductions est regroupée dans public/i18n.js ; les vues publiques comme les pages de contact et les écrans de l’application ne doivent pas introduire des chaînes isolées impossibles à traduire. Je traite la langue comme une propriété du produit, et pas comme un habillage ajouté en fin de développement.

L’internationalisation touche aussi les montants, les dates et les devises. Une devise affichée dans le tableau de bord ne doit jamais modifier les montants en centimes conservés dans les groupes. Le module currency-logic.js contrôle la date et les valeurs des taux, refuse une devise non reconnue et renvoie une absence de conversion si un taux requis manque. Ce comportement est plus rigoureux qu’un affichage de zéro ou qu’une conversion fictive.

Compteurs de notifications : le piège des faux nouveaux messages

notification-logic.js convertit différentes représentations de timestamps en millisecondes, ignore les actions rédigées par l’utilisateur lui-même et les éléments rejetés ou supprimés. Il ignore aussi les événements qui précèdent l’activation du suivi. Cette règle évite qu’un compteur « nouveau » apparaisse à tort lorsqu’un compte active ses notifications après une longue période d’utilisation.

Le badge se limite volontairement à une représentation « 4+ » lorsque le nombre dépasse quatre. Ce choix n'est pas anodin : il privilégie un signal d'attention court plutôt qu'un chiffre visuellement encombrant, tout en gardant le nombre exact manipulable dans le code.

La qualité d'un formulaire est liée aux états intermédiaires

Le formulaire de contact existant dispose d'un traitement asynchrone et d'un mécanisme d'expiration. Une action qui attend indéfiniment sans retour n'est pas un succès. La bonne interface présente les champs requis, empêche les envois invalides, indique le traitement en cours puis distingue clairement erreur technique et confirmation. Je préfère conserver une chaîne d'envoi éprouvée plutôt que la réécrire lors d'une refonte graphique.

> **Lecture produit**
>
> Une application multilingue cohérente doit maintenir les mêmes règles de calcul, d'autorisation et de validation dans toutes les langues. Le code de traduction ne doit pas être une seconde version de la logique métier.

19 Architecture de test : exemples reproductibles et limites

Tester une propriété, une transition et un accès

Je répartis les tests en trois familles. Les tests de propriétés vérifient les invariants mathématiques, par exemple la conservation des centimes. Les tests de transitions valident le passage d'un statut à un autre, par exemple la confirmation d'un règlement ou l'avancement d'une récurrence. Les tests d'accès cherchent à démontrer qu'un utilisateur non autorisé n'obtient pas ce que le frontend pourrait cacher.

tests/logic.test.js contient notamment des vérifications répétées de sommes de soldes et de propositions de transferts, avec des données générées. Un échantillon aléatoire est utile pour explorer des répartitions, mais il doit rester accompagné de cas fixes dont le résultat attendu est lisible par un humain. Sans cette combinaison, un test peut réussir sans démontrer les cas les plus risqués.

Constituer une matrice de couverture

Niveau	Exemple de test	Limite à reconnaître
Logique pure	Partage de 1 000 centimes entre 3 membres	Ne teste pas la base distante
Machine à états	Déclaration puis confirmation d'un paiement	Ne teste pas un vrai virement
Transaction simulée	Récurrence concurrente et redémarrage	Émulateur Firestore à ajouter
Contrôle statique	Présence d'un contrôle d'autorisation	Ne prouve pas une règle déployée
Navigateur	Inscription, navigation, viewport mobile	À refaire avec utilisateurs réels

Relier chaque régression à un test de non-retour

Un problème corrigé doit idéalement produire un test qui échoue avec l'ancien comportement et réussit avec la correction. Cela permet de conserver la raison de la modification dans l'outillage : l'équipe future n'a plus besoin de se souvenir oralement de tous les anciens incidents. C'est particulièrement important pour un projet qui a accumulé de nombreuses versions de design, de règles et de logique comptable.

Le workflow .github/workflows/checks.yml exécute les tests Node et le scanner de secrets sans accès à la production Firebase. Le contrôle est reproductible sur la branche, mais il ne réalise pas de déploiement, ni de test d'intégration sur un environnement cloud. J'explique cette frontière plutôt que de qualifier toutes les fonctionnalités de « vérifiées ».

20 Performance, volumes et coût d’exploitation

La performance n’est pas limitée au temps de rendu

Une application collaborative dépend des lectures en base, des abonnements en temps réel et du volume de données transféré. Un grand historique de messages ne doit pas nécessairement être écouté intégralement en permanence. Le code des conversations utilise des limitations de lecture ; la contrepartie est que les messages anciens peuvent ne plus déclencher certains comportements en direct sans pagination adaptée. Je préfère documenter cette limite plutôt que multiplier les lectures et augmenter les coûts.

Les opérations serveur doivent aussi rester contenues. Une récurrence qui rattraperait sans borne plusieurs années de données après une longue interruption pourrait produire une charge excessive. Le scénario de test documente une limite de douze échéances de rattrapage. Cette limite représente un arbitrage entre continuité fonctionnelle et stabilité du système.

Devises et services externes

Un appel de taux quotidien ne peut pas être considéré comme permanent ou toujours disponible. Le système doit distinguer taux valide, taux manquant, devise non supportée et réutilisation d’un taux antérieur. Surtout, une conversion de tableau de bord reste indicative : elle ne doit pas modifier le registre comptable du groupe.

La présence d’un backend Firebase apporte des fonctionnalités rapidement, mais elle implique une vigilance sur les quotas, le stockage, le trafic et la cohérence des règles entre environnements. Un projet techniquement correct sur un jeu de test peut se comporter autrement lorsque des centaines de groupes et de nombreux abonnements sont actifs.

Risque de charge	Indicateur à surveiller	Réponse prudente
Messages volumineux	Lectures et taille des flux	Pagination et bornage
Tâches récurrentes en retard	Nombre d’occurrences générées	Rattrapage limité
Devises externes	Âge du taux et erreurs API	Cache et état indisponible
Images et pièces jointes	Espace, bande passante, taille	Validation et quotas
Déploiement de règles	Erreurs de permission	Émulateurs et tests croisés

21 Journal de décisions : savoir défendre un compromis

Ce que les sources permettent de reconstruire

Un journal de décisions d'architecture ne prétend pas raconter jour par jour une histoire qui ne figure pas dans les commits. Il reformule les options possibles au moment où l'on étudie un choix observable. Je peux ainsi expliquer pourquoi Kortiz sépare ses modules calculatoires du navigateur, pourquoi le dashboard affiche des conversions sans les enregistrer et pourquoi le registre des paiements exige une confirmation.

La question universitaire pertinente est moins « quelle technologie ai-je utilisée ? » que « quelle contrainte ce choix résout-il, quel coût introduit-il et comment l'évaluer ? ». Une bibliothèque supplémentaire peut raccourcir une fonction tout en augmentant la dépendance ; un module JavaScript natif peut être simple à déployer tout en concentrant beaucoup de responsabilités dans un fichier volumineux.

Décision	Alternative envisagable	Justification et coût
Montants en centimes	Décimaux flottants	Exactitude; conversion explicite pour affichage
Soldes recalculés	Compteurs persistés	Traçabilité; lectures et calculs à maîtriser
Frontend JS natif	Refonte complète React	Simplicité actuelle; modularisation progressive souhaitable
Paiements hors app	Intégration bancaire	Périmètre légal et sécurité réduits; confirmation manuelle
Firebase en backend	Serveur autonome	Services intégrés; dépendance quotas et règles
Devises d'affichage	Comptabilité multi-devise	Ne pas modifier les dettes historiques

Un exercice de recul sur la maintenance

Une amélioration future pertinente serait de formaliser, pour chaque évolution sensible, un court enregistrement indiquant contexte, options, solution, critères de validation et stratégie de retour arrière. Cette méthode évite l'accumulation de corrections ponctuelles non documentées. Elle permet également de préparer une présentation orale fondée sur des preuves, sans attribuer à la chronologie des intentions impossibles à vérifier.

22 Préparer une soutenance technique

Question possible : Pourquoi avoir choisi de représenter l'argent en centimes ?

Réponse que je peux défendre : Parce qu'une application de partage doit conserver exactement les montants. Les entiers permettent de garantir la somme des parts et d'expliquer un centime attribué selon la méthode du plus fort reste.

Question possible : Quelle différence entre dépense, prévision et remboursement ?

Réponse que je peux défendre : Une dépense participe au solde réel, une prévision sert au budget sans créer de dette, un remboursement confirmé compense un solde. Une déclaration de paiement seule ne suffit pas.

Question possible : Pourquoi recalculer le solde depuis des écritures ?

Réponse que je peux défendre : Je conserve une trace interprétable des montants payés, dus et remboursés. Le recalcul rend visibles les conséquences des corrections et évite la dépendance exclusive à un compteur modifiable.

Question possible : Que se passe-t-il si un utilisateur falsifie un appel ?

Réponse que je peux défendre : La validation de la page n'est pas une permission. Les Cloud Functions vérifient l'authentification et l'appartenance ; les règles Firestore et Storage limitent l'accès aux ressources.

Question possible : Comment éviter les doublons sur une récurrence ?

Réponse que je peux défendre : Je combine des identifiants déterministes fondés sur l'occurrence et des transactions. Cela rend le traitement réessayable lorsqu'un déclenchement est répété.

Question possible : Qu'est-ce que 257 tests prouvent ?

Réponse que je peux défendre : Qu'une suite déterminée a été exécutée et a réussi sur le code testé. Elle ne démontre pas que tout le cloud et tous les appareils se comportent correctement.

Question possible : Pourquoi ne pas proposer de paiement dans l'application ?

Réponse que je peux défendre : Le besoin principal est la coordination et la preuve déclarative. Une intégration bancaire ajouterait des responsabilités réglementaires, techniques et de sécurité d'un tout autre ordre.

Question possible : Quelle est la prochaine amélioration prioritaire ?

Réponse que je peux défendre : Les tests d'intégration Firebase avec plusieurs comptes et un mécanisme de correction des écritures après remboursement confirmé.

23 Annexes : repères techniques et sources

Fichiers que je peux ouvrir pour démontrer un mécanisme

Source	Ce que j'y vérifie
public/app.js	parseAmountInput, allocate, splitAmount, computeBalances, minimalTransfers, IBAN, CSV.
public/currency-logic.js	Validation des taux et conversion d'affichage avec retour null si indisponible.
public/planning-logic.js	Dépenses réelles, prévisions, projection et budget.
public/i18n.js	Choix de langue, interpolation, traduction DOM et quatre langues.
functions/index.js	Invitations, contrôles de rôles, remboursements, récurrences, notifications, suppression.
functions/exit-debt-logic.js	Créances réservées lors des départs, soldes et transferts disponibles.
firestore.rules et storage.rules	Fonctions d'autorisation, accès à des données et aux médias.
tests/*.test.js	Calculs, sécurité, traductions, parcours sociaux et non-régression.
.github/workflows/checks.yml	Exécution CI des tests Node et du scanner de secrets.

Vocabulaire que je dois maîtriser

Terme	Définition dans le contexte de Kortiz
Invariant	Propriété qui doit rester vraie, par exemple la somme des parts.
Idempotence	Une opération rejouée ne produit pas plusieurs fois le même effet.
Autorisation	Décision vérifiée par le service sur les droits d'un compte.
Transaction	Ensemble d'écritures effectué de façon cohérente ou non appliqué.
Régression	Bug introduit par une modification d'un comportement précédemment correct.
Dette déclarative	Information de suivi, distincte d'une transaction bancaire.
Conversion FX	Conversion de présentation fondée sur des taux de change.
CI	Tests automatiquement lancés à l'occasion des changements de code.

Sources de référence et périmètre

Dépôt : https://github.com/carlxwj/kortiz ; documentation interne : README.md, docs/ARCHITECTURE.md, docs/TESTS_ET_QUALITE.md, docs/SECURITE_DONNEES.md, docs/DEPLOIEMENT.md, POLITIQUE_DEPENSES_ET_REMBOURSEMENTS.md. Sources techniques citées : fichiers publics du même dépôt préparés le 30 octobre 2026.

Le présent dossier est une présentation argumentée du code observé. Les problèmes classés comme « rencontrés » proviennent de comportements documentés et de garde-fous effectivement codés ; ce document ne reconstitue pas une chronologie personnelle complète à partir de journaux de développement inexistants. Les actions futures sont clairement présentées comme des perspectives et non des réalisations déjà validées.